



Università degli Studi di Padova

Laurea: Informatica

Corso: Ingegneria del Software

Anno Accademico: 2025/26



Gruppo 17

Nome: BitByBit

Email: swe.bitbybit@gmail.com

Specifica Tecnica



Stato: In lavorazione

Versione: 0.1.4

Responsabile: Gabriele Scaggiante

Redattori: Gabriele Scaggiante
Dennis Parolin
Ferdinando Fracasso

Verificatori: Riccardo Manisi
Dennis Parolin
Marco Sanguin

Destinatari: Miriade srl
Prof. Tullio Vardanega
Prof. Riccardo Cardin
Gruppo BitByBit



Registro delle modifiche

Versione	Data	Autore	Descrizione	Verificatore
0.1.7	2026-04-11	Marco Sanguin	Redatta la sezione Architettura funzionalità dead man switch	Riccardo Manisi
0.1.6	2026-04-09	Ferdinando Fracasso	Redatta la sezione Design pattern utilizzati	Giovanni Visentin
0.1.5	2026-04-06	Ferdinando Fracasso	Redatta la sezione Architettura funzionalità diari	Marco Sanguin
0.1.4	2026-04-05	Giovanni Visentin	Aggiunta della funzionalità dei contatti fidati	Dennis Parolin
0.1.3	2026-04-04	Gabriele Scaggianti	Iniziata la sezione Architettura frontend , completata Architettura chatbot	Marco Sanguin
0.1.2	2026-04-02	Dennis Parolin	Redatta la sezione Architettura logica e Architettura di deployment	Marco Sanguin
0.1.1	2026-03-31	Gabriele Scaggianti	Aggiunta di EventBridge, SAM, CloudFormation e Docker nella sezione tecnologie.	Dennis Parolin
0.1.0	2026-03-30	Gabriele Scaggianti	Creazione del documento. Scrittura dell'introduzione e delle tecnologie utilizzate.	Riccardo Manisi



Indice

1	Introduzione	8
1.1	Scopo del documento	8
1.2	Riferimenti	8
1.2.1	Riferimenti normativi	9
1.2.2	Riferimenti informativi	9
2	Tecnologie	10
2.1	Framework	10
2.1.1	Flutter	10
2.2	Linguaggi di programmazione	10
2.2.1	Dart	10
2.2.2	Python	11
2.3	Servizi AWS	11
2.3.1	Lambda	11
2.3.2	API Gateway	11
2.3.3	DynamoDB	11
2.3.4	S3	12
2.3.5	Cognito	12
2.3.6	Bedrock	12
2.3.7	SES	12
2.3.8	CloudWatch	12
2.3.9	EventBridge	13
2.3.10	SAM	13
2.3.11	CloudFormation	13
2.4	Librerie	13
2.4.1	http.dart	13
2.4.2	image_picker.dart	14
2.4.3	path_provider.dart	14
2.5	Tecnologie di analisi statica	14
2.5.1	SonarQube	14
2.6	Tecnologie di analisi dinamica	14
2.6.1	Pytest	14
2.6.2	Moto	15
2.7	Altro	15
2.7.1	Ollama	15
2.7.2	Docker	15

3	Architettura	15
3.1	Architettura logica	15
3.1.1	Livello Client (Frontend Mobile)	16
3.1.2	Livello di Servizio (Backend)	16
3.2	Design pattern utilizzati	17
3.2.1	Model-view-viewmodel	17
3.2.2	Observer	17
3.2.3	Proxy	18
3.2.4	Singleton	18
3.3	Architettura di deployment	18
3.3.1	Client Mobile	19
3.3.2	Infrastruttura AWS (Serverless Cloud)	19
3.3.3	Orchestrazione Event-Driven	20
3.3.4	Infrastructure as Code (IaC)	20
3.4	Architettura Front End	20
3.4.1	Chatbot	20
3.4.2	Diario criptato e diario fittizio	30
3.4.3	Contatti Fidati	38
3.4.4	Dead Man's Switch	43



Elenco delle tabelle

Elenco delle figure

1	Diagramma delle classi relativo alle funzionalità del Chatbot	21
2	Diagramma della classe ChatbotScreen	22
3	Diagramma della classe ChatWidget	22
4	Diagramma della classe ChatHistoryWidget	23
5	Diagramma della classe ChatbotModeToggleWidget	23
6	Diagramma della classe ChatbotSendMessageWidget	23
7	Diagramma della classe ChatbotCreateChatWidget	24
8	Diagramma della classe ChatbotViewModel	24
9	Diagramma della classe Chat	25
10	Diagramma della classe ProxyChat	25
11	Diagramma della classe LocalChat	26
12	Diagramma della classe ChatMessage	26
13	Diagramma della classe ChatMode	27
14	Diagramma della classe MessageType	27
15	Diagramma della classe MessageResponse	27
16	Diagramma della classe ChatbotRepository	28
17	Diagramma della classe ChatbotService	28
18	Diagramma della classe ChatDTO	29
19	Diagramma delle classi relativo alle funzionalità del diario criptato	30
20	Diagramma della classe DiaryAccessScreen	31
21	Diagramma della classe PasswordFormWidget	31
22	Diagramma della classe DiaryAccessViewModel	31
23	Diagramma della classe DiaryAccountRepository	32
24	Diagramma della classe DiaryAccountService	32
25	Diagramma della classe DiaryAccessResult	32
26	Diagramma della classe DiaryScreen	33
27	Diagramma della classe NoteListWidget	33
28	Diagramma della classe NoteActionsWidget	33
29	Diagramma della classe DiaryViewModel	34
30	Diagramma della classe DiarySession	34
31	Diagramma della classe DiaryType	34
32	Diagramma della classe Note	35
33	Diagramma della classe ProxyNote	35
34	Diagramma della classe LocalNote	36
35	Diagramma della classe NoteElement	36



36	Diagramma della classe NoteTextElement	36
37	Diagramma della classe NoteImgElement	37
38	Diagramma della classe NoteAudioElement	37
39	Diagramma della classe NoteService	37
40	Diagramma della classe NoteRepository	38
41	Diagramma della classe NoteDTO	38
42	Diagramma delle classi relativo alle funzionalità dei Contatti Fidati	39
43	Diagramma della classe TrustedContactScreen	40
44	Diagramma della classe TrustedContactListWidget	40
45	Diagramma della classe TrustedContactFormWidget	40
46	Diagramma della classe TrustedContactActionsWidget	41
47	Diagramma della classe TrustedContactViewModel	41
48	Diagramma della classe TrustedContact	42
49	Diagramma della classe TrustedContactRepository	42
50	Diagramma della classe TrustedContactService	43
51	Diagramma della classe TrustedContactDTO	43
52	Diagramma delle classi relativo alla funzionalità del Dead Man's Switch . .	44
53	Diagramma della classe DeadManScreen	45
54	Diagramma della classe DeadManFormWidget	45
55	Diagramma della classe DeadManViewModel	46
56	Diagramma della classe DeadManSettings	46
57	Diagramma della classe DeadManRepository	47
58	Diagramma della classe DeadManService	47
59	Diagramma della classe DeadManSettingsDTO	47



1. Introduzione

1.1. Scopo del documento

Il presente documento ha lo scopo di fornire una descrizione completa, formale e strutturata del prodotto software, delineandone in modo rigoroso le caratteristiche tecniche e architetturali.

La **Specifica Tecnica^G** rappresenta il riferimento principale per tutte le attività di progettazione, sviluppo, **Verifica^G** e manutenzione del sistema, assicurando coerenza rispetto ai requisiti definiti nel documento di **Analisi dei requisiti^G** e alle scelte progettuali adottate.

In particolare, il documento si propone di:

- Definire l'**Architettura^G** complessiva del sistema, descrivendo le componenti software, le loro responsabilità e le modalità di interazione attraverso appositi diagrammi
- Specificare i moduli funzionali e le interfacce esposte, includendo i contratti di comunicazione e i formati dei dati scambiati
- Descrivere l'**Architettura^G** di **Deployment^G**, evidenziando la distribuzione delle componenti nei diversi ambienti di esecuzione e le relative dipendenze infrastrutturali
- Documentare le tecnologie, i linguaggi di programmazione, i **Framework^G** e gli strumenti adottati, motivando le scelte effettuate in relazione ai requisiti di progetto e al **Capitolato^G**
- Illustrare i principali design pattern e le soluzioni architetturali utilizzate, con particolare attenzione agli aspetti di scalabilità, manutenibilità e sicurezza
- Fornire indicazioni sui metodi di testing e di **Validazione^G**
- Supportare le attività di evoluzione e manutenzione del software, garantendo una base documentale chiara e coerente.

1.2. Riferimenti

Al fine di evitare ambiguità relative al linguaggio e ai termini utilizzati all'interno dei documenti formali, viene allegato il [Glossario](#). I termini tecnici, gli acronimi e le parole con significato specifico all'interno del progetto sono contrassegnati da una lettera 'G' in apice (es. **Agile^G**) e sono descritti in dettaglio nel documento apposito.



1.2.1 Riferimenti normativi

- **Capitolato^G d'appalto C4: *L'app che Protegge e Trasforma***
 Parti consultate: documento completo
 Versione: 1.0
 Ultimo accesso: 2026-01-14
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C4.pdf>
- **Norme di Progetto**
 Parti consultate: documento completo
 Versione: v.1.0.0
 Ultimo accesso: 2026-02-09
https://swe-bitbybit.github.io/SWE-docs/docs/RTB/documenti_interni/norme_di_progetto.pdf

1.2.2 Riferimenti informativi

- **Slide sulla Progettazione Software del Prof. Vardanega**
 Parti consultate: documento completo
 Versione: A.A.2025/2026
 Ultimo accesso: 2026-03-30
<https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/T06.pdf>
- **Materiale relativo alla parte pratica del corso di Ingegneria del Software**
 Parti consultate: documento completo
 Versione: A.A.2022/2023
 Ultimo accesso: 2026-03-30
<https://www.math.unipd.it/~rcardin/swea/2022/>
- **Repository^G Github del Prof. Cardin**
 Ultimo accesso: 2026-03-30
<https://github.com/rcardin/swe-imdb>
- **Guida Flutter sull'Architettura^G di un'applicazione**
 Ultimo accesso: 2026-03-30
<https://docs.flutter.dev/app-architecture/guide>
- **Glossario**
 Parti consultate: documento completo
 Versione: v.1.0.0
 Ultimo accesso: 2026-02-27
https://swe-bitbybit.github.io/SWE-docs/docs/RTB/documenti_interni/Glossario.pdf



2. Tecnologie

Le tecnologie utilizzate sono state scelte trovando un punto d'incontro tra la necessità di garantire la sicurezza e la tutela dei dati personali degli utenti e il bisogno di realizzare un'**Architettura**^G solida, modulare e facilmente estendibile. Di seguito sono riportate tutte le tecnologie adottate

2.1. Framework

2.1.1 Flutter

Flutter è un **Framework**^G open-source sviluppato da Google per la creazione di applicazioni multi-piattaforma a partire da un unico codice sorgente. Consente di sviluppare interfacce per dispositivi mobili, web e desktop utilizzando il linguaggio Dart. Flutter si basa su un sistema di widget componibili, che possono rappresentare qualsiasi elemento dell'interfaccia **Utente**^G. Flutter include un proprio motore grafico (come Impeller o storicamente Skia) che disegna direttamente i componenti a schermo, garantendo elevata coerenza visiva e prestazioni indipendenti dalla piattaforma sottostante. È il **Framework**^G alla base dell'App Che Protegge e Trasforma, il che consente di poter generare build per iOS e Android da un unico codice sorgente, a fronte di un maggiore utilizzo di risorse rispetto alle soluzioni native e di una minore possibilità di controllo diretto e personalizzazione delle funzionalità native della piattaforma.

- **Versione utilizzata:** 3.41.6
- **Documentazione:** <https://docs.flutter.dev/>

2.2. Linguaggi di programmazione

2.2.1 Dart

Dart è un linguaggio di programmazione sviluppato da Google, progettato per la realizzazione di applicazioni client moderne. È un linguaggio tipizzato (con supporto sia statico che dinamico) e orientato agli oggetti. Viene utilizzato principalmente in combinazione con Flutter, dove il codice Dart viene compilato ahead-of-time in codice nativo per migliorare le prestazioni, oppure eseguito tramite compilazione just-in-time durante lo sviluppo. L'utilizzo di Flutter rende di fatto quasi obbligatoria l'adozione di Dart come linguaggio.

- **Versione utilizzata:** 3.11.4
- **Documentazione:** <https://dart.dev/docs>



2.2.2 Python

Python è un linguaggio di programmazione ad alto livello, interpretato e orientato agli oggetti, noto per la sua sintassi semplice e leggibile. Il codice Python viene eseguito da un interprete che traduce le istruzioni in bytecode, poi eseguito dalla macchina virtuale Python. Nel progetto è utilizzato lato **Backend^G** per la scrittura delle funzioni AWS Lambda e per i test tramite Pytest.

- **Versione utilizzata:** 3.14.3
- **Documentazione:** <https://docs.python.org/3/>

2.3. Servizi AWS

2.3.1 Lambda

AWS Lambda è un servizio **Serverless^G** che consente di eseguire codice senza gestire direttamente server o infrastruttura. Le funzioni vengono attivate in risposta a eventi e scalano automaticamente in base al carico. Nel progetto è utilizzato per implementare la logica **Backend^G**, eseguendo funzioni scritte in Python invocate tramite API Gateway.

- **Versione utilizzata:** Runtime Python 3.14
- **Documentazione:** <https://docs.aws.amazon.com/lambda/>

2.3.2 API Gateway

AWS API Gateway è un servizio che permette di creare, pubblicare e gestire **API REST^G** e HTTP, fungendo da punto di ingresso per le richieste client. Gestisce autenticazione, routing e integrazione con altri servizi AWS. Nel progetto è utilizzato per esporre endpoint HTTP che invocano le funzioni Lambda.

- **Versione utilizzata:** HTTP API (v2)
- **Documentazione:** <https://docs.aws.amazon.com/apigateway/>

2.3.3 DynamoDB

Amazon DynamoDB è un database NoSQL completamente gestito, progettato per offrire alte prestazioni e scalabilità automatica. I dati sono memorizzati in tabelle con chiavi primarie e accessibili con bassa latenza. Nel progetto è utilizzato per memorizzare dati strutturati come le note testuali del diario criptato e le chat con il chatbot.

- **Documentazione:** <https://docs.aws.amazon.com/dynamodb/>



2.3.4 S3

Amazon S3 (Simple Storage Service) è un servizio di storage a oggetti che consente di archiviare e recuperare dati in modo scalabile e sicuro. I file sono organizzati in bucket e accessibili tramite API. Nel progetto è utilizzato per memorizzare contenuti multimediali come immagini e file audio.

- **Documentazione:** <https://docs.aws.amazon.com/s3/>

2.3.5 Cognito

Amazon Cognito è un servizio di gestione delle identità che consente autenticazione, autorizzazione e gestione degli utenti. Supporta integrazione con provider esterni come Google. Nel progetto è utilizzato per gestire l'autenticazione degli utenti tramite account Google e per il controllo degli accessi.

- **Documentazione:** <https://docs.aws.amazon.com/cognito/>

2.3.6 Bedrock

Amazon Bedrock è un servizio che consente di utilizzare modelli di intelligenza artificiale generativa tramite API, senza gestire direttamente l'infrastruttura sottostante. Permette l'accesso a diversi modelli foundation per task come generazione di testo. Nel progetto è utilizzato per implementare la funzionalità di chatbot.

- **Documentazione:** <https://docs.aws.amazon.com/bedrock/>

2.3.7 SES

Amazon SES (Simple Email Service) è un servizio per l'invio e la ricezione di email in modo scalabile e affidabile. Supporta integrazione via API e SMTP. Nel progetto è utilizzato per inviare email ai contatti fidati degli utenti.

- **Documentazione:** <https://docs.aws.amazon.com/ses/>

2.3.8 CloudWatch

Amazon CloudWatch è un servizio di monitoraggio e osservabilità che raccoglie metriche, log ed eventi dai servizi AWS. Consente analisi, visualizzazione e configurazione di allarmi. Nel progetto è utilizzato per la raccolta e l'analisi dei log generati dalle funzioni Lambda.

- **Documentazione:** <https://docs.aws.amazon.com/cloudwatch/>



2.3.9 EventBridge

Amazon EventBridge è un servizio di event bus **Serverless^G** che consente di gestire e instradare eventi tra diversi servizi AWS. Esso permette di definire regole per reagire automaticamente a eventi generati da servizi AWS o da fonti esterne. Nel progetto è utilizzato per orchestrare i flussi di controllo di utilizzo dell'app da parte degli utenti, e l'invio delle email secondo le tempistiche stabilite dal Dead man's switch.

- **Documentazione:** <https://docs.aws.amazon.com/eventbridge/>

2.3.10 SAM

AWS **Serverless^G** Application Model (SAM) è un **Framework^G** open-source che semplifica la definizione e il **Deployment^G** di applicazioni **Serverless^G** su AWS. Estende AWS CloudFormation introducendo una sintassi più compatta per risorse come Lambda, API Gateway e DynamoDB e include strumenti per build, testing locale e deploy delle applicazioni. Nel progetto è utilizzato per definire e distribuire l'infrastruttura **Serverless^G** in modo semplificato.

- **Documentazione:** <https://docs.aws.amazon.com/serverless-application-model/>

2.3.11 CloudFormation

AWS CloudFormation è un servizio di **Infrastructure as Code^G** che consente di modellare e gestire risorse AWS tramite template dichiarativi in formato **YAML^G** o JSON. Nel progetto è utilizzato come base per il provisioning dell'infrastruttura, anche tramite template generati da SAM.

- **Documentazione:** <https://docs.aws.amazon.com/cloudformation/>
- **Versioni:** CloudFormation non prevede versioni di servizio esplicite; eventuali aggiornamenti sono gestiti direttamente da AWS. I template possono però utilizzare versioni di specifiche risorse e trasformazioni (es. AWS::**Serverless^G**).

2.4. Librerie

2.4.1 http.dart

http è una libreria per il linguaggio Dart che consente di effettuare richieste HTTP in modo semplice e asincrono. Fornisce metodi per eseguire operazioni come GET, POST, PUT e DELETE, gestendo automaticamente la serializzazione delle richieste e la ricezione delle risposte. È progettata per essere leggera e facilmente integrabile in applicazioni client.

- **Versione utilizzata:** 1.6.0
- **Documentazione:** <https://pub.dev/packages/http>



2.4.2 image_picker.dart

image_picker è una libreria Flutter che permette di accedere alla fotocamera e alla galleria del dispositivo per selezionare o acquisire immagini e video. Fornisce un'interfaccia unificata tra piattaforme diverse, gestendo le differenze tra iOS e Android.

- **Versione utilizzata:** 1.2.1
- **Documentazione:** https://pub.dev/packages/image_picker

2.4.3 path_provider.dart

path_provider è una libreria Flutter che consente di ottenere i percorsi delle directory del file system del dispositivo, come directory temporanee o documenti applicativi. Facilita l'accesso a percorsi corretti e compatibili tra piattaforme per la gestione dei file.

- **Versione utilizzata:** 2.1.5
- **Documentazione:** https://pub.dev/packages/path_provider

2.5. Tecnologie di analisi statica

2.5.1 SonarQube

SonarQube è una piattaforma open-source per l'analisi continua della qualità del codice. Esegue **Analisi Statica**^G su diversi linguaggi di programmazione per individuare bug, vulnerabilità, code smells e problemi di copertura dei test. Il funzionamento si basa su scanner che analizzano il codice sorgente e inviano i risultati a un server centrale, dove vengono aggregati e visualizzati tramite una dashboard dedicata.

- **Versione utilizzata:** 26.3.0
- **Documentazione:** <https://docs.sonarsource.com/sonarqube/>

2.6. Tecnologie di analisi dinamica

2.6.1 Pytest

Pytest è un **Framework**^G di testing per il linguaggio Python che consente di scrivere test in modo semplice e scalabile. Supporta test funzionali, unitari e di integrazione, offrendo funzionalità come fixture, parametrizzazione dei test e scoperta automatica dei test.

- **Versione utilizzata:** 9.0.2
- **Documentazione:** <https://docs.pytest.org/>



2.6.2 Moto

Moto è una libreria Python che consente di mockare i servizi AWS durante i test. Simula il comportamento delle API AWS, permettendo di testare codice che interagisce con servizi **Cloud^G** senza effettuare chiamate reali. Funziona intercettando le richieste e restituendo risposte simulate coerenti con quelle dei servizi AWS.

- **Versione utilizzata:** 5.1.23
- **Documentazione:** <https://docs.getmoto.org/>

2.7. Altro

2.7.1 Ollama

Ollama è una piattaforma che consente di eseguire e gestire LLM in locale tramite una semplice interfaccia. Permette il download, l'esecuzione e l'interazione con modelli di intelligenza artificiale direttamente in ambiente locale, senza necessità di servizi **Cloud^G** esterni. Il funzionamento si basa su un runtime che gestisce i modelli e espone endpoint per inferenza.

- **Versione utilizzata:** 0.18.3
- **Documentazione:** <https://ollama.com/docs>

2.7.2 Docker

Docker è una piattaforma di containerizzazione che consente di creare, distribuire ed eseguire applicazioni all'interno di ambienti isolati (container). I container includono tutto il necessario per eseguire un'applicazione (codice, runtime, librerie e dipendenze), garantendo portabilità e consistenza tra ambienti diversi. Nel progetto è utilizzato a supporto delle pipeline di **CI/CD^G** e per testare in locale servizi come DynamoDB e S3.

- **Documentazione:** <https://docs.docker.com/>

3. Architettura

3.1. Architettura logica

Il sistema software è progettato secondo i principi di un'**Architettura a Strati^G** (o *Layered Architecture*), che impone una chiara separazione delle responsabilità tra i diversi livelli applicativi del sistema. L'obiettivo cardine di questa scelta è massimizzare l'indipendenza formale della logica di business dalle interfacce grafiche **Utente^G** e dai meccanismi fisici di gestione del dato, favorendo così la facilità di test, la manutenzione nel tempo e il forte disaccoppiamento (*Separation of Concerns^G*).



All'ingresso superiore dell'applicazione si posiziona l'ecosistema logico del dispositivo, mentre le fondamenta sono governate e calcolate dall'infrastruttura di servizio. L'interfaccia tra questi due mondi è ben definita tramite API e totalmente slegata dai linguaggi di programmazione o dai sistemi operativi utilizzati.

3.1.1 Livello Client (Frontend Mobile)

Il client mobile è sviluppato incapsulando il codice secondo i dettami moderni di una **Clean Architecture^G** (fortemente affine al noto pattern architetturale *Model-View-ViewModel* - MVVM). Il codice dell'applicazione è suddiviso in almeno tre strati logici indipendenti e gerarchici:

- **Presentation Layer (View)**: Costituisce il livello più visibile e frontale dell'applicazione. Riceve gli input da parte dell'**Utente^G** e si occupa unicamente di aggregare e renderizzare il layout grafico a schermo tramite widget componenti, senza mai eseguire logica complessa.
- **State Management^G Layer (ViewModel)**: Funge da mediatore tra View e dominio, gestendo lo stato della view. Riceve input dalla view, muta lo stato dell'UI e notifica la view che si aggiornerà sulla base del nuovo stato incapsulato (es. passando da "In caricamento" a "Successo"). Inoltre, gestisce le validazioni elementari dei dati inseriti, comandando ai servizi sottostanti le operazioni formali da compiere.
- **Data Layer^G**: È lo strato più basso dell'ambiente Client, incaricato unicamente dell'acquisizione e gestione dei dati, ed è logicamente suddiviso in *Service* e *Repository^G*. I Service sono gli unici componenti ad interfacciarsi con l'esterno, conoscendo gli endpoint del **Backend^G** per eseguire le chiamate asincrone HTTP e gestendo l'accesso a dati memorizzati localmente o in remoto. I **Repository^G** fungono da intermediari: raccolgono i dati grezzi provenienti dai Service e li formattano in modelli di dominio strutturati (es. un oggetto *Note*), fornendoli al ViewModel puliti e pronti all'uso.

3.1.2 Livello di Servizio (Backend)

L'**Architettura^G** logica adibita all'interazione tra i device mobili e l'ambiente computazionale in **Cloud^G** si fonda su un analogo approccio modulare disaccoppiato in tre sotto-livelli principali:

- **Presentation e Routing Layer**: Costituisce il *Single Point of Entry^G* dell'infrastruttura esposto al software. Ha la pura e unica responsabilità di tracciare e smistare le richieste esterne verso i rami corretti. Intercetta validazioni e autorizzazioni per poi rilasciare formalmente il **Payload^G** agli strati contigui.



- **Business Core / Logic Layer:** Costituisce il cuore elaborativo del sistema, dove risiede l'intera *Business Logic*^G. Questo strato è implementato tramite unità funzionali indipendenti e distribuite (*Funzioni Lambda*) ed è progettato per essere completamente agnostico rispetto ai dettagli di trasporto e comunicazione (come il protocollo HTTP). Il suo compito esclusivo è ricevere i dati già validati dal livello di *Routing*, applicare le regole di dominio, coordinare l'elaborazione degli eventi asincroni e restituire le informazioni elaborate e strutturate.
- **Data Access Layer**^G: Un livello totalmente passivo e disaccoppiato che offre un'astrazione tramite moduli e **Driver**^G. Consente una persistenza dati logica e mascherata ai database sottostanti (quali storage documentali, array non relazionali o conservazione audio/video grezza), celando alla logica di business i layer fisici o i linguaggi di interrogazione diretti della persistenza **Cloud**^G.

3.2. Design pattern utilizzati

3.2.1 Model-view-viewmodel

Descrizione del pattern: Il pattern Model-view-viewmodel (MVVM) prevede la separazione della parte software responsabile per la resa grafica dalle parti responsabili della **Presentation Logic**^G e **Business Logic**^G in tre componenti principali:

- **Model:** contiene il codice responsabile per la **Business Logic**^G dell'applicazione.
- **View:** contiene la parte responsabile esclusivamente per la resa grafica del software, e delega la gestione dell'interazione **Utente**^G alla componente *ViewModel*.
- **ViewModel:** contiene il codice responsabile per la gestione della **Presentation Logic**^G e per la coordinazione tra i componenti *View* e *Model*.

Ragioni per l'utilizzo: L'utilizzo del pattern permette una più semplice gestione delle varie parti dell'applicazione, dato che limita notevolmente il coupling tra UI e logica sottostante, migliorandone quindi la manutenibilità, e rende lo sviluppo degli unit test più semplice.

3.2.2 Observer

Descrizione del pattern: Il pattern Observer definisce la capacità per un oggetto *Publisher* di definire un meccanismo di "iscrizione" per cui un insieme di oggetti *Subscriber* può "osservarlo" ed essere notificato solo in seguito ad un suo cambiamento di stato.

Ragioni per l'utilizzo: L'uso principale del pattern nell'applicazione è nelle classi *ViewModel*, dato che permette di notificare solo i widget che richiedono modifiche dopo un cambiamento di stato del *ViewModel*, invece di dover ricostruire la schermata dopo



una qualsiasi interazione, permettendo così una maggiore fluidità dell'interfaccia durante l'uso da parte dell'**Utente^G**. Inoltre il meccanismo di "iscrizione" rimuove la necessità di verifiche frequenti sullo stato del *Publisher* da parte dei *Subscriber* e rende il codice più facilmente estendibile e mantenibile dato che la modifica di una componente non va a influire sulle altre, per esempio, l'aggiunta di un widget alla *View* non richiederebbe modifiche al *ViewModel*.

3.2.3 Proxy

Descrizione del pattern: Il pattern Proxy permette la creazione di un oggetto *Proxy* che va a controllare l'accesso all'oggetto reale. L'oggetto *Proxy* deve implementare la stessa interfaccia dell'oggetto reale, in modo da essere sempre utilizzabile quando il codice si aspetta l'oggetto reale, e una volta ricevuta una richiesta deve delegarla ad una istanza dell'oggetto reale.

Ragioni per l'utilizzo: L'utilizzo del pattern Proxy per le strutture dati delle classi **Chat** e **Note** permette di rimandare la creazione di un loro oggetto fino a quando non effettivamente necessario, evitando così la creazione di tutti gli oggetti e dei loro componenti in una singola volta quando vengono inizialmente caricati, così risparmiando memoria che altrimenti sarebbe sprecata con oggetti pesanti.

3.2.4 Singleton

Descrizione del pattern: Il pattern Singleton permette di avere la garanzia che in un dato momento esista al più una singola istanza della classe a cui è applicato. Questo si ottiene facendo in modo che il costruttore dell'oggetto ne crei uno nuovo solamente se non ne esiste già una copia, altrimenti ritorna un puntatore alla copia esistente.

Ragioni per l'utilizzo: Nell'applicazione questo pattern viene utilizzato per garantire l'esistenza di una sola istanza della classe **DiarySession** in un dato momento, in modo che l'**Utente^G** abbia accesso ad un solo versione del diario, criptato o fittizio, alla volta, evitando la condivisione accidentale dei dati tra le due versioni.

3.3. Architettura di deployment

L'**Architettura^G** di **Deployment^G** del sistema si divide in due componenti principali: l'applicazione Client (eseguita sul dispositivo dell'**Utente^G**) e l'infrastruttura di **Backend^G** (eseguita in **Cloud^G**). Il sistema si basa su un paradigma **Serverless^G** ed **Event-Driven^G**, per garantire scalabilità, elevata disponibilità e abbattimento dei costi legati alla gestione manuale dei server.



3.3.1 Client Mobile

L'applicazione viene rilasciata come eseguibile nativo sui dispositivi mobili degli utenti (Android/iOS). Essendo sviluppata tramite il **Framework^G Flutter**, il codice scritto in Dart viene compilato in formato binario specifico per l'**Architettura^G** hardware del dispositivo ospite. Il Client si interfaccia in modo del tutto indipendente ai servizi di **Backend^G** unicamente tramite la rete Internet, sfruttando protocolli di comunicazione sicuri.

3.3.2 Infrastruttura AWS (Serverless Cloud)

L'intero ecosistema di **Backend^G** è ospitato e gestito all'interno di **Amazon Web Services (AWS)**. Questa scelta architetturale consente:

- **Scalabilità dinamica^G**: le risorse allocate (il calcolo e lo storage) scalano o si accendono/spengono automaticamente in base al volume di richieste ricevute dai Client;
- **Isolamento e Sicurezza**: ogni funzione ha una specifica responsabilità e policy di accesso minima verso le altre risorse, limitando l'esposizione al minimo indispensabile;
- **Gestione del servizio (Zero Ops^G)**: l'uso di infrastrutture completamente gestite e la persistenza dei dati automatizzata tramite **Amazon DynamoDB** e **Amazon S3** libera il team dagli oneri di manutenzione sistemistica.

Il flusso operativo del sistema e la connessione tra i componenti AWS si articolano nei seguenti layer di **Deployment^G**:

- **Amazon API Gateway**: costituisce l'unico punto di ingresso espositivo e si occupa di orchestrare e bilanciare le richieste HTTP provenienti dal Client.
- **Amazon Cognito**: agisce come filtro primario in stretta collaborazione con l'API Gateway, validando i token **Utente^G** e negando le richieste non autorizzate.
- **AWS Lambda**: rappresenta il cuore elaborativo decentralizzato; l'infrastruttura crea dinamicamente container di esecuzione che accolgono le richieste smistate e processano la **Business Logic^G** interna.
- **Amazon DynamoDB e Amazon S3**: compongono lo strato di persistenza profondo ospitando, in alta disponibilità, i dati relazionali/JSON e i contenuti multimediali grezzi sollevando il livello server della necessità di memoria sul disco.



3.3.3 Orchestrazione Event-Driven

Oltre alle comunicazioni sincrone HTTP per le API *REST*, l'**Architettura^G** di **Deployment^G** prevede un'orchestrazione asincrona fondamentale per le funzionalità dell'applicazione (es. le notifiche del sistema *Dead Man's Switch*). I componenti comunicano internamente tramite eventi gestiti da **Amazon EventBridge**, che funge da **Message Bus^G** e permette:

- **Comunicazione asincrona e disaccoppiata:** riducendo la dipendenza operativa tra i servizi;
- L'esecuzione ritardata e pianificata di check sui dati salvati nel database;
- L'instradamento di Eventi critici e allerte verso il servizio **Amazon SES** per l'invio automatizzato di e-mail.

3.3.4 Infrastructure as Code (IaC)

Il **Deployment^G** materiale di tutte le risorse in ambiente AWS non è eseguito manualmente, ma viene automatizzato in modo riproducibile attraverso l'impiego di **AWS SAM** (**Serverless^G Application Model**) e **AWS CloudFormation**. L'infrastruttura logica e fisica è versionata sotto forma di file **YAML^G**, aderendo interamente al paradigma dell'*Infrastructure as Code^G*.

3.4. Architettura Front End

3.4.1 Chatbot

La funzionalità del chatbot permette all'**Utente^G** di comunicare via chat con un modello di intelligenza artificiale (LLM) e di memorizzare e caricare tutte le chat passate. La funzionalità del chatbot è implementata secondo il pattern MVVM e le linee guida architetturali di Flutter, distinguendo quindi un **Data Layer^G**, composto da Service e **Repository^G**, e un **UI Layer^G**, composto da View e ViewModel. Tra questi layer figurano i vari domain models: le classi Chat (**ProxyChat** e **LocalChat**), **ChatMessage** e **MessageResponse**.





ChatbotScreen La classe **ChatbotScreen** rappresenta la schermata dell'applicazione dedicata al Chatbot all'interno del *Presentation Layer*. La classe è composta a sua volta da numerosi widget e contiene la minima logica necessaria al rendering di tali componenti visivi, permettendo quindi una netta separazione tra presentazione e logica secondo il pattern *MVVM*. Tra i numerosi widget che la costituiscono, i principali sono **ChatWidget**, **ChatHistoryWidget**, **ChatbotModeToggleWidget** e **ChatbotSendMessageWidget**.

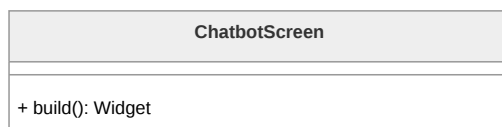


Figura 2: Diagramma della classe **ChatbotScreen**

ChatWidget La classe **ChatWidget** è responsabile della visualizzazione della conversazione attiva. In quanto **Consumer**, essa osserva il **ChatbotViewModel** per ottenere i dati aggiornati e reagisce ai cambiamenti dello stato applicativo, garantendo la costruzione automatica dei widget rappresentanti i messaggi scambiati in chat.

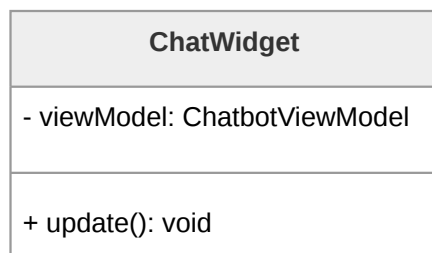


Figura 3: Diagramma della classe **ChatWidget**

ChatHistoryWidget **ChatHistoryWidget** si occupa di mostrare l'elenco delle conversazioni passate sotto forma di anteprime. Tale widget è inserito all'interno di un menu a tendina dedicato, e, in quanto **Consumer**, osserva il **ChatbotViewModel** per recuperare i dati aggiornati relativi alle chat passate. Questo widget trae grande beneficio dall'utilizzo del pattern Proxy per il fetch delle chat. Tale pattern consente infatti di generare la lista di anteprime delle conversazioni passate senza dover recuperare l'intero contenuto di ogni singola chat.

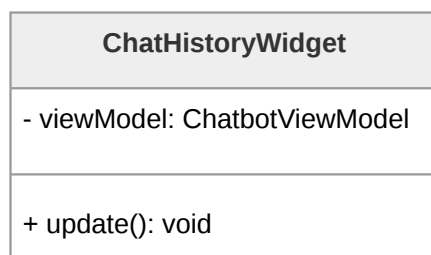


Figura 4: Diagramma della classe **ChatHistoryWidget**

ChatbotModeToggleWidget Questo componente dell'interfaccia consente all'**Utente^G** di selezionare la modalità operativa del chatbot. Le due modalità disponibili sono il Detective delle Relazioni e lo Specchio Intelligente. La modalità in uso viene specificata nell'invocazione dei metodi relativi all'invio di un messaggio in chat e determinano come il chatbot risponde ai messaggi dell'**Utente^G**. La generazione della risposta è a completo carico del **Backend^G**.

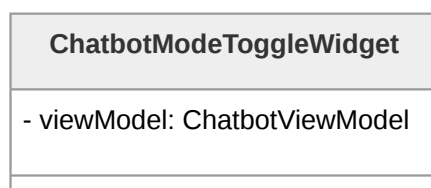


Figura 5: Diagramma della classe **ChatbotModeToggleWidget**

ChatbotSendMessageWidget **ChatbotSendMessageWidget** è un pulsante che gestisce l'invio dei messaggi da parte dell'**Utente^G**, delegando la gestione dei nuovi messaggi al ViewModel. Quando **ChatbotSendMessageWidget** viene premuto, il messaggio contenuto nel campo d'inserimento testuale viene inoltrato a **ChatbotViewModel**, che a sua volta comunica con **ChatbotRepository** in modo da memorizzare il messaggio nella chat corrente e di ricevere la risposta dal LLM.

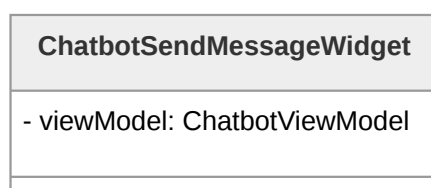


Figura 6: Diagramma della classe **ChatbotSendMessageWidget**



ChatbotCreateChatWidget Questo widget fornisce all'**Utente^G** la possibilità di avviare una nuova conversazione. Anche in questo caso, la logica viene delegata al View-Model, preservando il disaccoppiamento tra interfaccia e dominio applicativo.

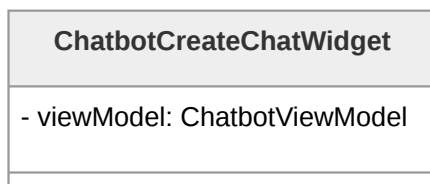


Figura 7: Diagramma della classe **ChatbotCreateChatWidget**

ChatbotViewModel **ChatbotViewModel** gestisce lo stato dell'interfaccia chatbot: mantiene la lista delle conversazioni passate, la chat correntemente aperta e la modalità operativa selezionata. In quanto **ChangeNotifier**, notifica automaticamente le classi View (**ChatWidget**, **ChatHistoryWidget** e i widget di azione) ogni volta che lo stato cambia. Per accedere ai dati si affida a **ChatbotRepository**, al quale delega il recupero e la persistenza delle conversazioni.

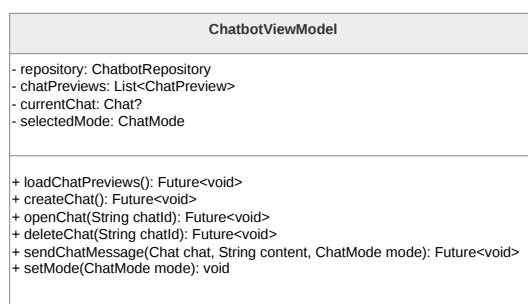
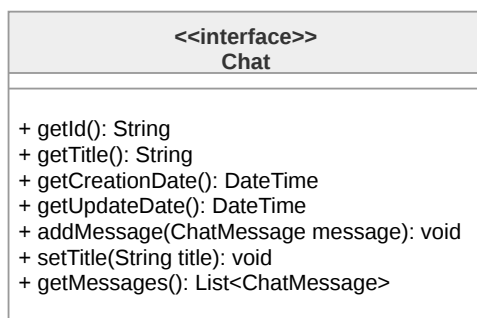
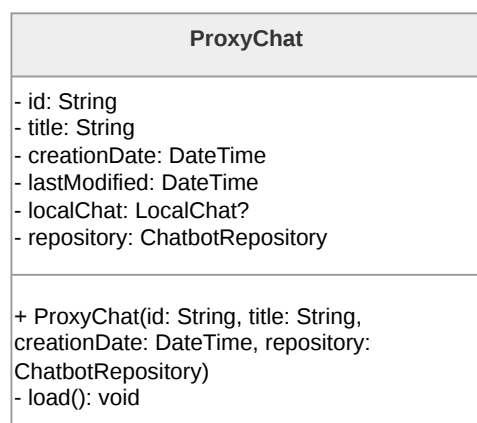


Figura 8: Diagramma della classe **ChatbotViewModel**

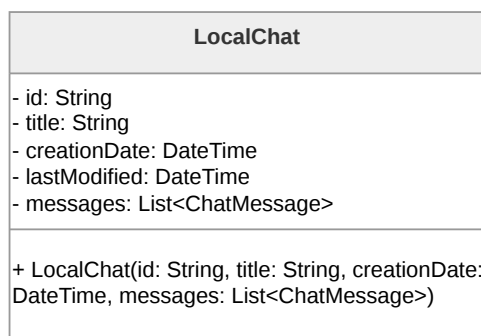
Chat L'interfaccia **Chat** definisce il modello astratto di una conversazione tra **Utente^G** e il chatbot. Insieme a **ProxyChat** e **LocalChat**, è utilizzata nell'implementazione del pattern Proxy.

Figura 9: Diagramma della classe **Chat**

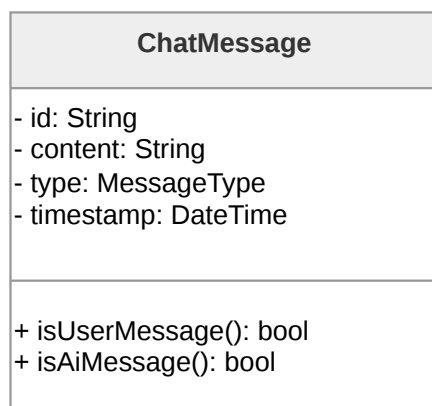
ProxyChat **ProxyChat** implementa l'interfaccia **Chat** e viene utilizzata da **ChatbotViewModel** per rappresentare le conversazioni storiche. Ritarda il caricamento completo dei messaggi, delegandolo a **ChatbotRepository** solo quando necessario, e mantiene internamente un riferimento a **LocalChat**, che istanzia e popola al primo accesso effettivo ai dati.

Figura 10: Diagramma della classe **ProxyChat**

LocalChat **LocalChat** è l'implementazione del modello di conversazione che mantiene in memoria tutte le informazioni e i messaggi associati alla chat. Essa rappresenta l'oggetto reale utilizzato da **ProxyChat** quando c'è effettivamente bisogno di caricare i messaggi della chat, ad esempio per visualizzarli a schermo.

Figura 11: Diagramma della classe **LocalChat**

ChatMessage **ChatMessage** modella un singolo messaggio all'interno di una conversazione, includendo contenuto, tipologia e data di creazione e di aggiornamento. Fornisce inoltre un modo per distinguere tra messaggi generati dall'**Utente^G** e quelli prodotti dal LLM.

Figura 12: Diagramma della classe **ChatMessage**

ChatMode **ChatMode** è un'enumerazione che rappresenta le due diverse modalità operative del chatbot. Viene utilizzato da **ChatbotViewModel**, che lo inoltra a **ChatbotRepository**, il quale lo utilizza per invocare i metodi del **ChatbotService** relativi all'invio dei messaggi in chat. Sarà poi il **Backend^G** a interpretarlo e utilizzarlo per generare la risposta al messaggio inviato dall'**Utente^G**.

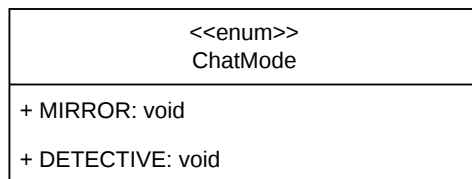


Figura 13: Diagramma della classe **ChatMode**

MessageType **MessageType** è un'enumerazione che consente di classificare i messaggi in base alla loro origine, distinguendo tra input dell'**Utente^G** e output del LLM. È utilizzata nella classe **ChatMessage** e necessaria per stabilire l'impaginazione dei messaggi all'interno di **ChatWidget**.

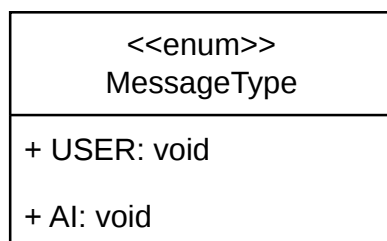


Figura 14: Diagramma della classe **MessageType**

MessageResponse **MessageResponse** incapsula il risultato di un'interazione con il LLM, includendo il messaggio generato e, eventualmente, il titolo generato a partire dal messaggio originale. È utilizzato da **ChatbotRepository**, che a sua volta lo ritorna in risposta a **ChatbotViewModel** per l'aggiornamento dello stato della View.

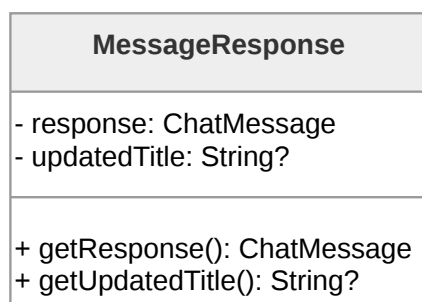


Figura 15: Diagramma della classe **MessageResponse**



ChatbotRepository **ChatbotRepository** è il tramite tra **ChatbotViewModel** e la comunicazione di rete: riceve le richieste dal ViewModel, le inoltra a **ChatbotService** e trasforma le risposte grezze in oggetti di dominio (**LocalChat**, **ChatMessage**) tramite **ChatDTO**, restituendoli al ViewModel in un formato pronto all'uso.

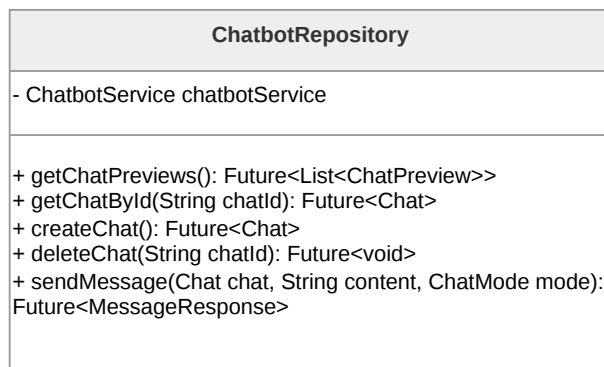


Figura 16: Diagramma della classe **ChatbotRepository**

ChatbotService **ChatbotService** rappresenta il livello infrastrutturale responsabile della comunicazione con le API remote. Si occupa della gestione delle richieste e delle risposte, operando su rappresentazioni dei dati adatte allo scambio su rete. Restituisce i dati grezzi a **ChatbotRepository**, il quale si occupa poi di trasformarli in modelli di dominio.

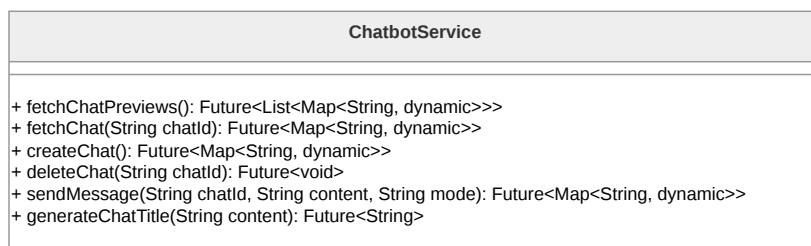


Figura 17: Diagramma della classe **ChatbotService**

ChatDTO **ChatDTO** è un oggetto di trasferimento dati utilizzato per convertire le informazioni tra il formato utilizzato dal sistema esterno e quello del dominio applicativo. Ha due ruoli principali:

- trasformare i risultati delle chiamate ad API da JSON a oggetti di tipo **Chat**
- serializzare oggetti di tipo **Chat**, generando la loro rappresentazione JSON per l'invio alle API.

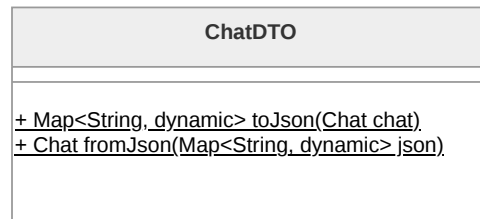


Figura 18: Diagramma della classe **ChatDTO**



DiaryAccessScreen La classe **DiaryAccessScreen** si occupa della rappresentazione visiva della schermata per l'accesso alla funzionalità dei diari. Contiene solamente la logica necessaria per il rendering dei widget che la compongono, tra cui **PasswordFormWidget**, per garantire la separazione tra presentazione e logica, come previsto dal pattern MVVM. Per questo delega la gestione dello stato a **DiaryAccessViewModel**.

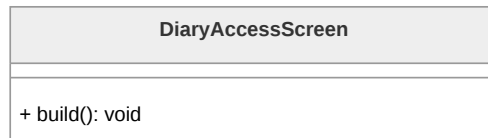


Figura 20: Diagramma della classe **DiaryAccessScreen**

PasswordFormWidget La classe **PasswordFormWidget** è responsabile della visualizzazione dei campi utilizzati per l'accesso ai diari. **PasswordFormWidget** è un **Consumer** che osserva **DiaryAccessViewModel** per ottenere i dati aggiornati e reagire di conseguenza.

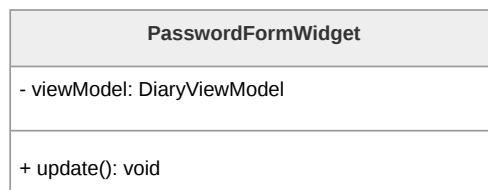


Figura 21: Diagramma della classe **PasswordFormWidget**

DiaryAccessViewModel La classe **DiaryAccessViewModel** gestisce lo stato dell'applicazione e coordina le operazioni tra i Layer, implementando il pattern Observer. la classe è un **ChangeNotifier** e mantiene una lista di *listeners* che vengono notificati quando avvengono cambiamenti che necessitano di un loro aggiornamento, in questo caso componenti che devono reagire a tentativi di accesso ai diari da parte dell'**Utente^G**. Per accedere ai dati e verificare le credenziali si affida alla classe **DiaryAccountRepository**.

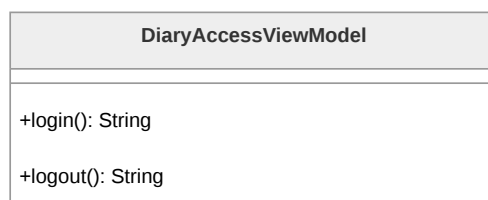


Figura 22: Diagramma della classe **DiaryAccessViewModel**



DiaryAccountRepository `DiaryAccountRepository` si occupa della trasformazione delle risposte ricevute da `DiaryAccountService` in oggetti di dominio e nascondendone la natura al resto dell'applicazione, fungendo da strato di astrazione tra il *service* ed il ViewModel.

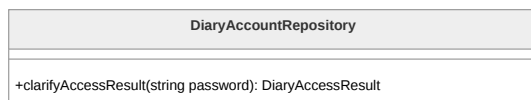


Figura 23: Diagramma della classe `DiaryAccountRepository`

DiaryAccountService La classe `DiaryAccountService` rappresenta lo strato infrastrutturale responsabile della comunicazione con le API remote, occupandosi della gestione delle richieste e delle risposte. Restituisce i dati grezzi a `DiaryAccountRepository`, il quale si occupa poi di trasformarli in oggetti di dominio.

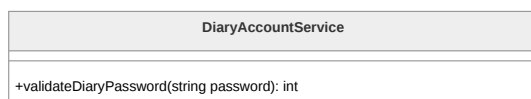


Figura 24: Diagramma della classe `DiaryAccountService`

DiaryAccessResult L'enumerazione `DiaryAccessResult` permette la classificazione della risposta ad un tentativo di accesso ai diari da parte dell'**Utente^G**, distinguendo tra accesso avvenuto con successo ad uno dei due tipi di diario, e accesso fallito con errore o superamento del limite di tentativi di accesso.

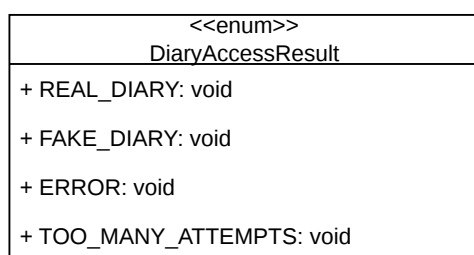


Figura 25: Diagramma della classe `DiaryAccessResult`

DiaryScreen La classe `DiaryScreen` gestisce la rappresentazione visiva della schermata principale dei diari nel *presentation layer*, contenendo solamente la logica necessaria al rendering dei suoi componenti widget, tra cui `NoteListWidget` e `NoteActionsWidget`. La gestione dello stato è delegata alla classe `DiaryViewModel`.

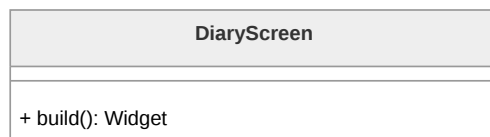


Figura 26: Diagramma della classe **DiaryScreen**

NoteListWidget La classe **NoteListWidget** si occupa della visualizzazione della lista delle note contenute nel diario in cui l'**Utente^G** ha effettuato l'accesso e l'eventuale visualizzazione dei contenuti di una nota selezionata. Trattandosi di un **Consumer**, la classe osserva gli aggiornamenti dei dati in **DiaryViewModel** in modo da reagire di conseguenza alla selezione, modifica o eliminazione delle note.

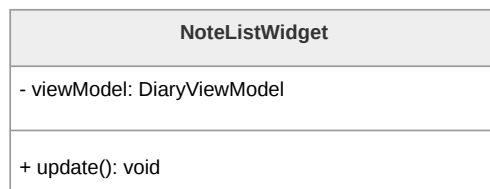


Figura 27: Diagramma della classe **NoteListWidget**

NoteActionsWidget La classe **NoteActionsWidget** si occupa della visualizzazione delle componenti di interazione della funzionalità di diario. La logica applicativa viene delegata a **DiaryViewModel**, preservando il disaccoppiamento tra interfaccia e dominio applicativo.

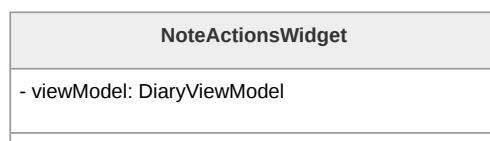
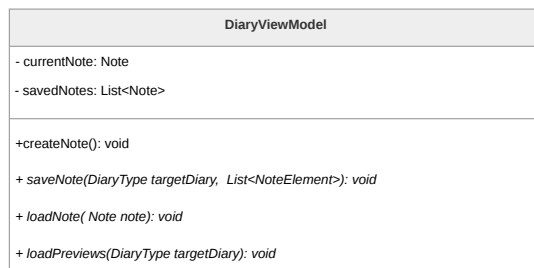


Figura 28: Diagramma della classe **NoteActionsWidget**

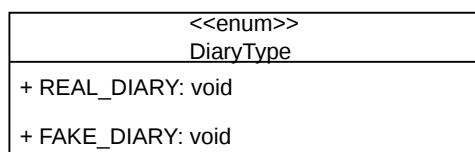
DiaryViewModel **DiaryViewModel** si occupa della gestione dello stato dell'applicazione e del coordinamento delle operazioni tra UI e **Data Layer^G**. La classe tiene conto della lista di note presenti all'interno del diario attualmente aperto e della nota attualmente aperta dall'**Utente^G**. Inoltre delega alla classe **NoteRepository** l'accesso e la gestione dei dati delle note per le operazioni di memoria. La classe implementa il Mixin **ChangeNotifier**, e quindi il pattern Observer, mantenendo una lista di *listeners* che vengono notificati a seguito di operazioni a loro rilevanti, in questo caso **NoteListWidget** e **NoteActionsWidget**.

Figura 29: Diagramma della classe **DiaryViewModel**

DiarySession La classe **DiarySession** modella l'istanza effettiva del diario, tenendo traccia del tipo di diario a cui l'**Utente^G** ha effettuato l'accesso e gestendo le informazioni di sessione. La classe implementa il pattern Singleton per fare in modo che in ogni dato momento di utilizzo dell'applicazione possa esistere al più una singola istanza di diario.

Figura 30: Diagramma della classe **DiarySession**

DiaryType L'enumerazione **DiaryType** rappresenta le due tipologie di diario disponibili e viene utilizzata da **DiarySession** per comunicare a **DiaryViewModel** quale archivio di note visualizzare.

Figura 31: Diagramma della classe **DiaryType**



Note L'interfaccia **Note** definisce i metodi necessari per gli oggetti rappresentanti le note all'interno dei diari. Implementa il pattern Proxy insieme alle classi **ProxyNote** e **LocalNote**.

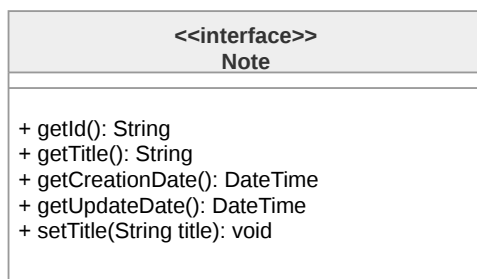


Figura 32: Diagramma della classe **Note**

ProxyNote La classe **ProxyNote** realizza il pattern Proxy per rendere più efficiente il caricamento da **Repository^G** delle note, permettendo di rimandare il caricamento dei dati interni di una nota fino a quando non è effettivamente necessario, e così risparmiando risorse di sistema. La classe contiene un riferimento ad un oggetto **LocalNote** a cui delega le operazioni una volta effettuato il caricamento della nota.

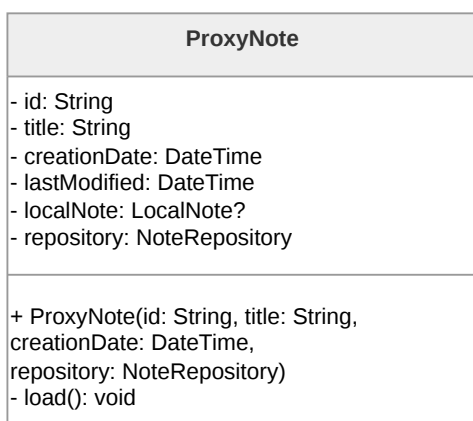


Figura 33: Diagramma della classe **ProxyNote**

LocalNote **LocalNote** è l'implementazione dell'interfaccia **Note** che rappresenta l'oggetto reale, utilizzato dall'implementazione proxy per le operazioni sulla nota una volta effettuato il suo caricamento.

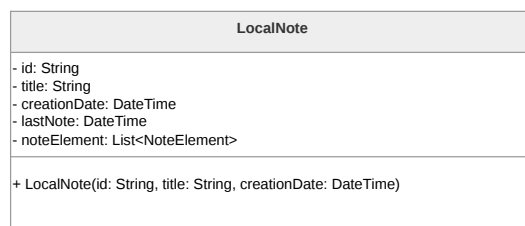


Figura 34: Diagramma della classe **LocalNote**

NoteElement La classe astratta **NoteElement** rappresenta un elemento presente all'interno di una nota e ha delle specializzazioni in base al tipo di contenuto dell'oggetto, che può essere testuale, un'immagine oppure del contenuto audio.

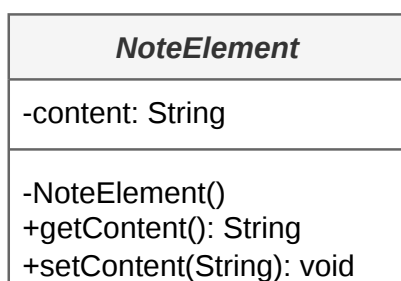


Figura 35: Diagramma della classe **NoteElement**

NoteTextElement La classe concreta **NoteTextElement** è la specializzazione di **NoteElement** per contenuti testuali.

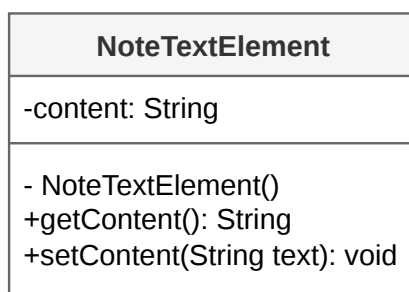


Figura 36: Diagramma della classe **NoteTextElement**



NoteImgElement La classe concreta **NoteImgElement** è la specializzazione di **NoteElement** per contenuti di tipo grafico.

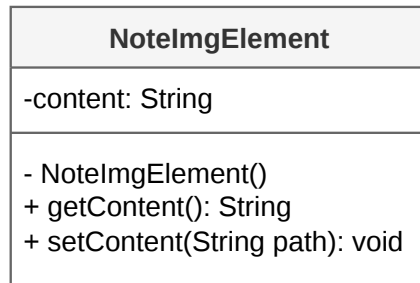


Figura 37: Diagramma della classe **NoteImgElement**

NoteAudioElement La classe concreta **NoteAudioElement** è la specializzazione di **NoteElement** per contenuti audio.

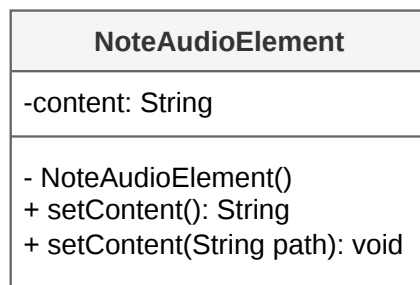


Figura 38: Diagramma della classe **NoteAudioElement**

NoteService La classe **NoteService** rappresenta il livello infrastrutturale dedicato alla comunicazione con le API remote, gestendo richieste e risposte durante le operazioni sulle note. Restituisce i dati grezzi a **NoteRepository**, il quale si occupa poi di trasformarli in modelli di dominio tramite **NoteDTO**.

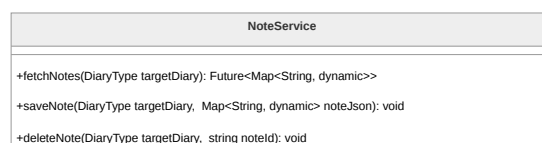


Figura 39: Diagramma della classe **NoteService**



NoteRepository La classe **NoteRepository** funge da strato di astrazione tra **DiaryViewModel** e la **Persistence Logic^G** di **NoteService**, trasformando i dati grezzi in oggetti di dominio tramite **NoteDTO** e nascondendone la natura al resto dell'applicazione.

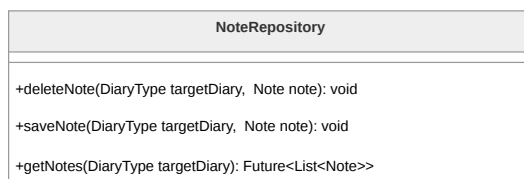


Figura 40: Diagramma della classe **NoteRepository**

NoteDTO **NoteDTO** è un Data Transfer Object con due ruoli principali:

- deserializzare i risultati delle chiamate API, in formato JSON, trasformandoli in oggetti di tipo **Note**.
- serializzare oggetti di tipo **Note**, generando la loro rappresentazione JSON per operazioni che richiedono l'invio dei dati alle API, come il salvataggio di una nuova nota.

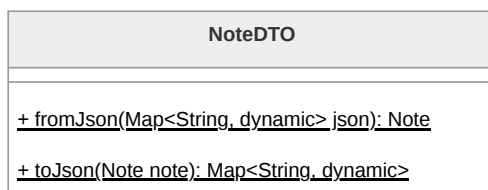


Figura 41: Diagramma della classe **NoteDTO**

3.4.3 Contatti Fidati

La funzionalità dei Contatti Fidati permette all'**Utente^G** di gestire una lista di persone di fiducia da avvisare in caso di inattività tramite il sistema di *Dead Man's Switch*. Analogamente al resto dell'applicazione, la funzionalità è implementata secondo il pattern MVVM e le linee guida architetturali di Flutter, distinguendo un **Data Layer^G**, composto da Service e **Repository^G**, e un **UI Layer^G**, composto da View e ViewModel.

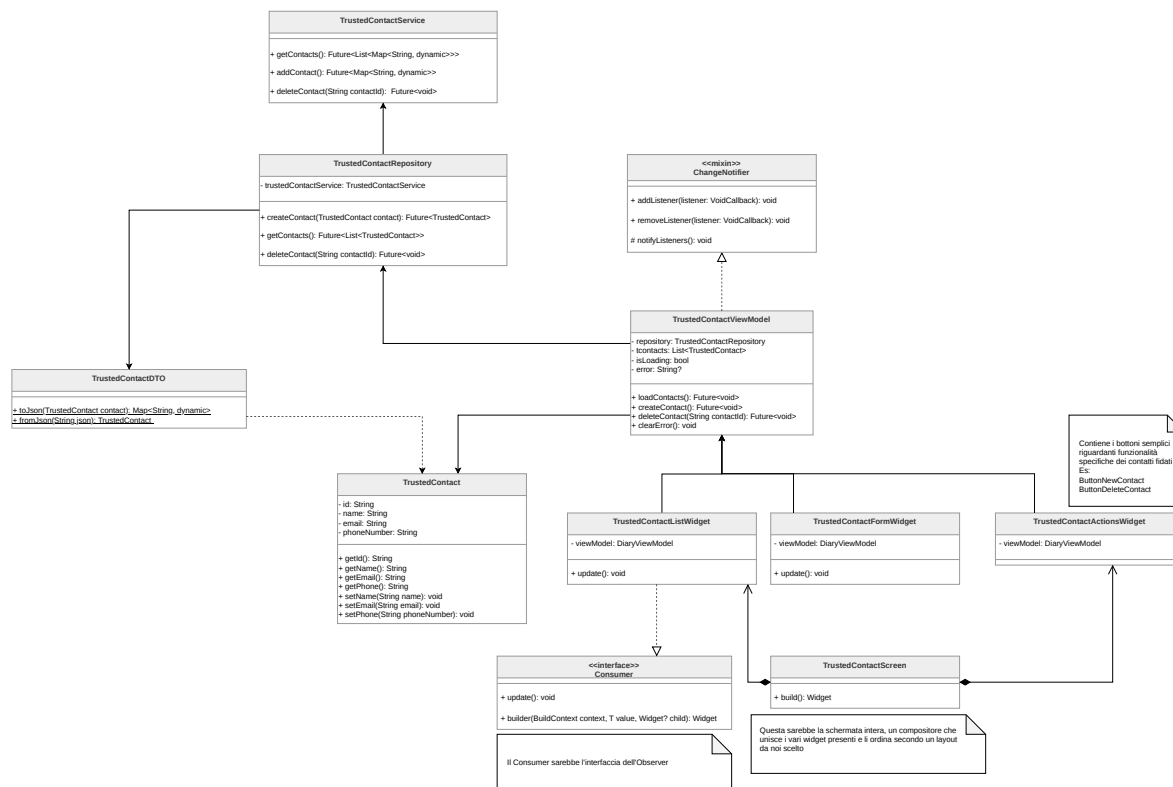


Figura 42: Diagramma delle classi relativo alle funzionalità dei Contatti Fidati



TrustedContactScreen La classe **TrustedContactScreen** rappresenta la schermata principale dedicata alla gestione dei contatti fidati all'interno del *Presentation Layer*. Essa funge da compositore, unendo i vari widget presenti e ordinandoli secondo il layout stabilito, delegando la logica di stato ai componenti sottostanti.

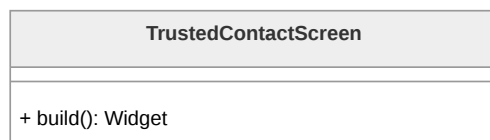


Figura 43: Diagramma della classe **TrustedContactScreen**

TrustedContactListWidget La classe **TrustedContactListWidget** è responsabile della visualizzazione dell'elenco dei contatti attualmente salvati. Implementando l'interfaccia **Consumer**, osserva il ViewModel per reagire ai cambiamenti di stato e aggiornare dinamicamente l'interfaccia grafica.



Figura 44: Diagramma della classe **TrustedContactListWidget**

TrustedContactFormWidget **TrustedContactFormWidget** gestisce i campi di input necessari per l'inserimento o la modifica dei dati di un contatto fidato. In quanto **Consumer**, si aggiorna in base allo stato del ViewModel.

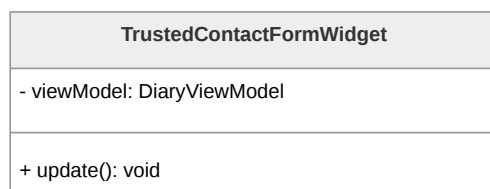


Figura 45: Diagramma della classe **TrustedContactFormWidget**

TrustedContactActionsWidget Questo widget raggruppa i bottoni e le azioni specifiche per la gestione dei contatti fidati (ad esempio i pulsanti per l'aggiunta di un nuovo contatto o l'eliminazione di uno esistente). Comunica le intenzioni dell'**Utente^G** direttamente al ViewModel.



Figura 46: Diagramma della classe **TrustedContactActionsWidget**

TrustedContactViewModel **TrustedContactViewModel** è il nucleo della gestione dello stato per questa funzionalità. Mantiene la lista dei contatti (**tcontacts**), lo stato di caricamento (**isLoading**) e la gestione degli errori (**error**). Espone i metodi per caricare, creare e cancellare i contatti, facendo da ponte tra la View e il **Repository^G**. Utilizza il mixin **ChangeNotifier** per notificare i widget in ascolto.

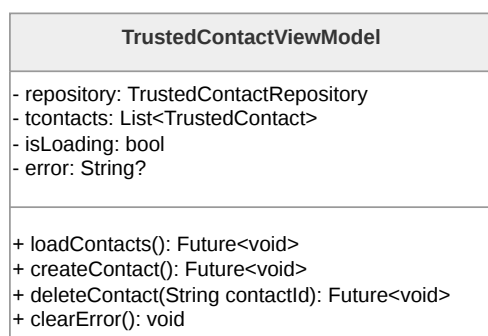


Figura 47: Diagramma della classe **TrustedContactViewModel**

TrustedContact **TrustedContact** è la classe di dominio che modella il singolo contatto fidato. Contiene le informazioni anagrafiche e di recapito essenziali, quali l'identificativo unico, il nome, l'indirizzo email e il numero di telefono, fornendo i relativi metodi di accesso e modifica.

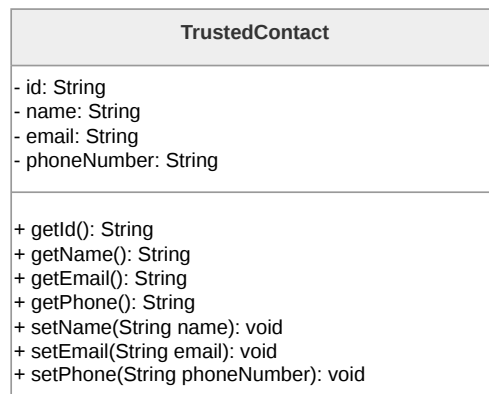


Figura 48: Diagramma della classe **TrustedContact**

TrustedContactRepository **TrustedContactRepository** astrae la sorgente dei dati per il ViewModel. Utilizza **TrustedContactService** per eseguire le operazioni CRUD (creazione, lettura, eliminazione), trasformando i dati ricevuti in oggetti di dominio **TrustedContact** e isolando la logica di business dai dettagli di implementazione del network.

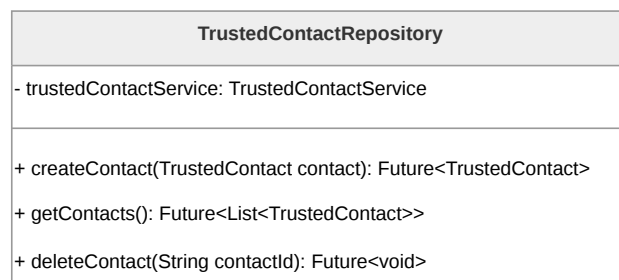


Figura 49: Diagramma della classe **TrustedContactRepository**

TrustedContactService **TrustedContactService** appartiene al livello infrastrutturale e si occupa di effettuare fisicamente le chiamate API verso il **Backend^G**. Gestisce le richieste HTTP per ottenere l'elenco dei contatti, aggiungerne di nuovi o eliminarli, restituendo le risposte grezze al **Repository^G**.

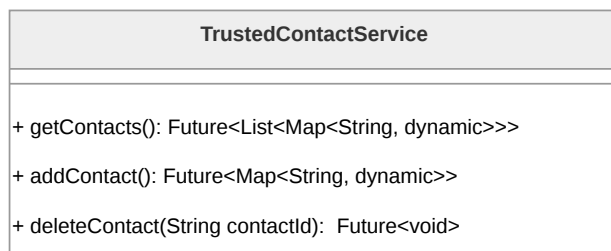


Figura 50: Diagramma della classe **TrustedContactService**

TrustedContactDTO **TrustedContactDTO** (*Data Transfer Object*) gestisce la serializzazione e deserializzazione dei dati. Mappa i dati JSON provenienti dall'esterno in oggetti **TrustedContact** e viceversa, garantendo la compatibilità del formato scambiato con le API.

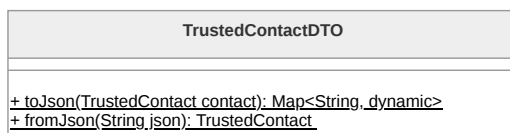


Figura 51: Diagramma della classe **TrustedContactDTO**

3.4.4 Dead Man's Switch

La funzionalità del *Dead Man's Switch* (Interruttore dell'Uomo Morto) rappresenta un sistema di sicurezza critico che invia avvisi automatici in caso di inattività prolungata dell'**Utente^G**. L'**Architettura^G** frontend è focalizzata sulla gestione sicura della configurazione di tale sistema. Il frontend non interagisce direttamente con servizi di posta elettronica, ma demanda la gestione dei timer e l'invio delle email tramite AWS SES al **Backend^G**, garantendo così la sicurezza e l'affidabilità del servizio anche ad applicazione chiusa. Analogamente al resto dell'applicazione, la funzionalità è implementata secondo il pattern MVVM e le linee guida architetturali di Flutter, distinguendo un **Data Layer^G**, composto da Service e **Repository^G**, e un **UI Layer^G**, composto da View e ViewModel.

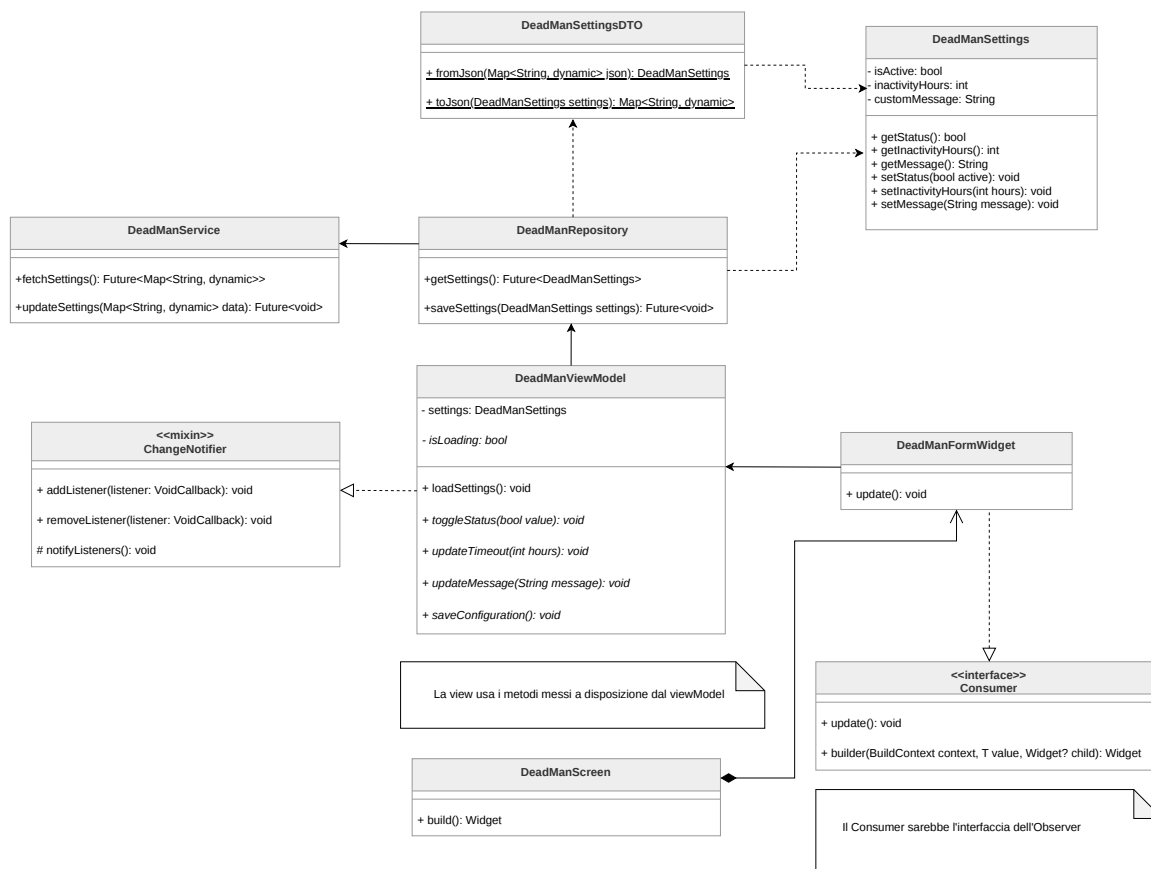


Figura 52: Diagramma delle classi relativo alla funzionalità del Dead Man's Switch



DeadManScreen La classe **DeadManScreen** rappresenta la schermata principale del pannello di controllo per la sicurezza. Funge da compositore, strutturando il layout della pagina e ospitando i form interattivi, delegando al ViewModel la reattività e la logica di salvataggio.

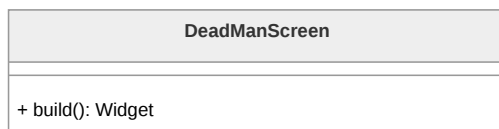


Figura 53: Diagramma della classe **DeadManScreen**

DeadManFormWidget Questo componente incapsula gli elementi interattivi dell'interfaccia, come i selettori (switch) per l'attivazione del servizio, i campi di input per la soglia di inattività (es. ore o giorni) e le aree di testo per il messaggio di emergenza personalizzato. Implementando **Consumer**, aggiorna i propri campi riflettendo fedelmente lo stato mantenuto nel ViewModel.



Figura 54: Diagramma della classe **DeadManFormWidget**

DeadManViewModel **DeadManViewModel** orchestra la logica di business di questa sezione. Mantiene in memoria le configurazioni attuali dell'**Utente^G** (**settings**) e lo stato di sincronizzazione (**isLoading**). Implementa il mixin **ChangeNotifier** per notificare la UI a ogni interazione. Oltre a recuperare i dati, espone metodi fondamentali come **saveConfiguration()**, che permette di confermare e trasmettere al **Repository^G** le nuove impostazioni scelte dall'**Utente^G**.

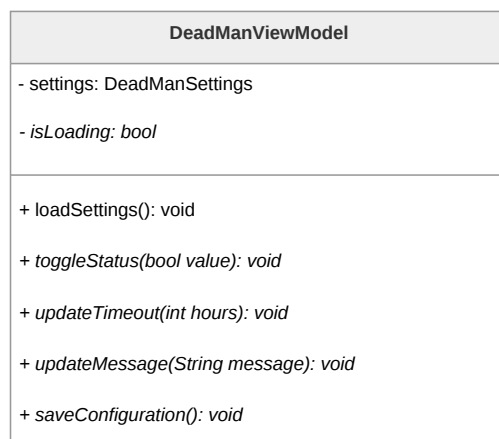


Figura 55: Diagramma della classe **DeadManViewModel**

DeadManSettings **DeadManSettings** è l'entità di dominio che modella le preferenze di sicurezza. Contiene gli attributi necessari a configurare l'allarme: uno stato booleano di attivazione (**isActive**), un valore temporale che definisce la soglia di tolleranza (**inactivityHours**) e il corpo del messaggio testuale (**customMessage**) che il **Backend^G** inoltrerà ai contatti fidati tramite AWS SES.

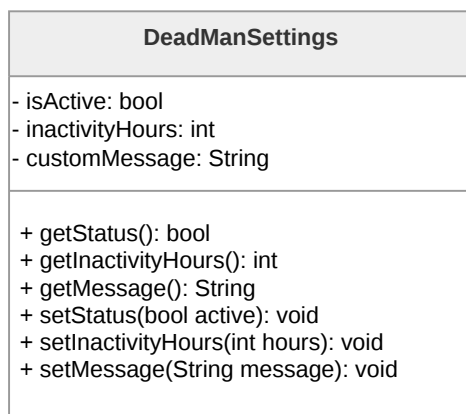


Figura 56: Diagramma della classe **DeadManSettings**

DeadManRepository Il **DeadManRepository** gestisce la persistenza e il recupero delle configurazioni. Agisce da ponte tra la logica di presentazione e il livello infrastrutturale, utilizzando **DeadManService** per inviare le preferenze aggiornate e convertendo le risposte di rete in oggetti di dominio **DeadManSettings** sicuri e validati tramite il DTO.

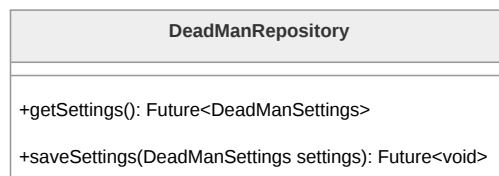


Figura 57: Diagramma della classe **DeadManRepository**

DeadManService Appartenente al **Data Layer^G**, **DeadManService** si occupa della comunicazione HTTP/REST con le API del **Backend^G**. Dispone dei metodi necessari per effettuare le operazioni di *GET* (lettura parametri) e *PUT/POST* (aggiornamento configurazioni) sul database remoto.

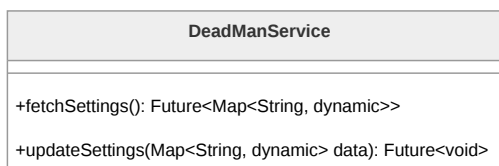


Figura 58: Diagramma della classe **DeadManService**

DeadManSettingsDTO Il **DeadManSettingsDTO** è l'oggetto di trasferimento dati incaricato della serializzazione. Consente di mappare i campi JSON inviati e ricevuti dal server, assicurando che la comunicazione tra l'app Flutter e il **Backend^G** mantenga un **Contratto^G** dati rigoroso.

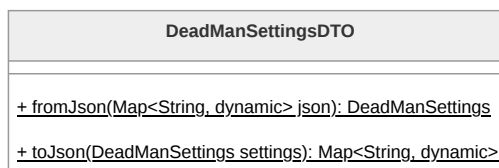


Figura 59: Diagramma della classe **DeadManSettingsDTO**